

Sistema de Conciliación ISO 20022

Documentación técnica completa — Agente de IA con LangGraph

PACS.008 · CAMT.053 · PACS.002 · LangGraph · PostgreSQL · Docker · React

Python 3.11	LangGraph	Claude Sonnet	ISO 20022	PostgreSQL	Redis	FastAPI	React	Docker
-------------	-----------	---------------	-----------	------------	-------	---------	-------	--------

Resumen del sistema

Este documento describe el sistema completo de conciliación automática de pagos ISO 20022, construido como un agente de IA con LangGraph y Claude. El sistema concilia mensajes **PACS.008** (instrucciones de pago) contra **CAMT.053** (extractos bancarios), incorpora el procesamiento de rechazos vía **PACS.002**, aplica fuzzy matching por embeddings vectoriales como último recurso, y expone una API REST consumida por un dashboard React para la operación diaria del equipo de tesorería.

Métrica	Valor	Referencia manual
Tasa de autoconciliación	> 95%	60–70%
Tiempo por lote (500 pagos)	< 30s	2–4 horas
Motores de matching	3 en cascada	1 (manual)
Audit trail	100% trazable	Parcial
Tiempo de despliegue	1 comando	Días

Índice de contenidos

1.	Arquitectura del sistema
2.	Módulos Python — descripción y responsabilidades
3.	Agente LangGraph — grafo y nodos
4.	Motor de matching — tres estrategias en cascada
5.	Scheduler — triggers y deduplicación
6.	Base de datos PostgreSQL — schema y audit trail
7.	API del dashboard — endpoints
8.	Frontend React — dashboard operativo

9. Infraestructura Docker — despliegue en producción

10

- . Operación y comandos del día a día
-

11

- . Roadmap y próximos pasos
-

Generado el 19 de May de 2026 a las 10:16 UTC

1. Arquitectura del sistema

Visión de alto nivel de las 15 capas del sistema completo

El sistema está compuesto por 15 archivos organizados en 7 capas funcionales. Cada capa tiene una responsabilidad única y puede evolucionar de forma independiente.

Capa 1 — Parser ISO 20022

`iso20022_parser.py` · `pacs002_handler.py`

Convierte XML crudo a dataclasses Python normalizados. Soporta PACS.008, CAMT.053 y PACS.002. El resto del sistema nunca toca XML directamente.

Capa 2 — Motores de matching

`agente_conciliacion.py` · `fuzzy_matching_embeddings.py`

Motor 1: matching exacto por UETR/EndToEndId. Motor 2: rechazos PACS.002. Motor 3: fuzzy embeddings vectoriales. Los tres determinísticos excepto el análisis LLM.

Capa 3 — Agente LangGraph

`agente_completo.py`

Grafo de estado que orquesta los tres motores en cascada. El LLM (Claude Sonnet) actúa como orquestador y analista de excepciones. Expone API FastAPI en puerto 8000.

Capa 4 — Scheduler

`scheduler.py`

Tres triggers: cron diario, webhook bancario y watcher SFTP. Deduplicación por hash SHA-256 en Redis. Notificaciones Slack. Puerto 8001.

Capa 5 — Persistencia

`database.py` · `schema_postgresql.sql`

5 tablas PostgreSQL con triggers automáticos. Audit trail inmutable. Vistas analíticas para el dashboard. Función `guardar_resultado_conciliacion()`.

Capa 6 — API dashboard

`api_dashboard.py`

9 endpoints REST para el frontend: KPIs, excepciones, resolución, audit trail, trazabilidad de pagos y health check detallado.

Capa 7 — Frontend y despliegue

`Dashboard.jsx` · `docker-compose.yml` · `Dockerfile` · `nginx.conf`

Dashboard React con KPIs en tiempo real, tabla de excepciones, modal de resolución y gráfico histórico. Docker Compose levanta todo en 1 comando.

2. Módulos Python

15 archivos — responsabilidad y dependencias de cada uno

Archivo	Responsabilidad	Importa de
iso20022_parser.py	Parser PACS.008 y CAMT.053 → dataclasses	lxml, decimal, datetime
pacs002_handler.py	Parser PACS.002, códigos de rechazo, accioneslxml, iso20022_parser	
agente_conciliacion.py	Motor matching exacto (UETR/E2E/fuzzy)	iso20022_parser, langchain
fuzzy_matching_embeddings.py	Embeddings vectoriales, similitud coseno	numpy, sklearn, anthropic
agente_completo.py	Grafo LangGraph + 5 tools + API FastAPI	todos los anteriores
api_dashboard.py	9 endpoints REST para el dashboard React	database, fastapi
scheduler.py	APScheduler + webhook + watcher SFTP	httpx, redis, fastapi
database.py	AuditRepository: guardar y consultar runs	sqlalchemy, json
schema_postgresql.sql	5 tablas + triggers + vistas analíticas	(SQL puro)
Dashboard.jsx	UI React: KPIs, excepciones, resolución	react, tailwind
docker-compose.yml	Orquestación: postgres, redis, agente, scheduler, nginx	(YAML)
Dockerfile	Multi-stage: deps → dev → producción	(Docker)
nginx.conf	Reverse proxy, rate limiting, rutas seguras	(nginx)
requirements.txt	Todas las dependencias Python con versiones	pip
.env.example / Makefile	Configuración y comandos de operación	(shell)

3. Agente LangGraph

El grafo de estado que orquesta los tres motores

El agente usa un único bucle LLM → herramienta → LLM hasta que Claude decide que ha completado los 5 pasos del protocolo. Esto permite que el LLM salte pasos cuando no son necesarios (por ejemplo, si no hay PACS.002 o si no quedan SIN_MATCH).

Nodo	Tipo	Descripción
inicializar	Python puro	Genera run_id, abre audit_trail, inicializa estado
llamar_llm	Claude Sonnet	Razona sobre el estado actual y decide qué herramienta llamar
ejecutar_herramientas	Python puro	Ejecuta las tool_calls del LLM, actualiza estado
escribir_audit	Python puro	Persiste en PostgreSQL via AuditRepository
END	LangGraph	Fin del grafo

Las 5 herramientas del agente

Herramienta	Cuándo se llama	Devuelve
parsear_mensajes	Paso 1 — siempre	instrucciones + movimientos + estados_pacs002
ejecutar_motor_exacto	Paso 2 — siempre	resultados con CONCILIADO/EXCEPCION/SIN_MATCH
aplicar_estados_pacs002	Paso 3 — si hay PACS.002	resultados con RECHAZADO/EN_TRANSITO actualizados
ejecutar_motor_embeddings	Paso 4 — solo si hay SIN_MATCH	candidatos fuzzy + resultados actualizados
analizar_y_generar_reporte	Paso 5 — siempre al final	resumen + acciones + importe conciliado/pendiente

Principio de diseño: el motor de matching es 100% determinístico — el LLM nunca toma decisiones de match/no-match. Solo analiza los resultados para generar narrativa y acciones. Esto garantiza reproducibilidad y cumplimiento regulatorio.

4. Motor de matching

Tres estrategias en cascada — de más a menos precisa

Cada instrucción PACS.008 pasa por las estrategias en orden. El motor se detiene en la primera que tenga éxito. Solo llega a embeddings si las dos primeras fallan.

Estrategia	Criterios	Estado resultado	Confianza
1. UETR exacto	UETR == UETR Importe idéntico	CONCILIADO	99.9%
2. EndToEndId exacto	EndToEndId == EndToEndId Importe idéntico Fecha idéntica	CONCILIADO	95%
3. EndToEndId fuzzy	EndToEndId == EndToEndId Delta importe <= EUR 1,00 Delta fecha <= 2 días	EXCEPCION_PARCIAL	Variable
4. PACS.002 rechazo	EndToEndId en PACS.002 TxSts == RJCT	RECHAZADO	100%
5. PACS.002 aceptado	EndToEndId en PACS.002 TxSts == ACSC/ACSP	EN_TRANSITO	100%
6. Embeddings vectoriales	Similitud coseno >= 0.82 Delta importe <= 0.5% Delta fecha <= 5 días	MATCH_FUZZY	Ver similitud
7. Sin match	Ninguna estrategia anterior	SIN_MATCH	—

Códigos de rechazo PACS.002 más frecuentes

Código	Descripción	Acción requerida
AC01	IBAN del beneficiario incorrecto	Verificar IBAN y reenviar
AC04	Cuenta cancelada	Obtener nueva cuenta del beneficiario
AM04	Fondos insuficientes	Reponer fondos y reenviar
AM05	Pago duplicado detectado	Verificar si el original fue procesado
TM01	Corte de tiempo superado	Reenviar al día siguiente antes de las 10:00
RC01	BIC del banco incorrecto	Verificar BIC con el beneficiario
LEGL	Restricción legal / compliance	Escalar a compliance inmediatamente
FF01	Formato del mensaje incorrecto	Revisar el XML PACS.008 generado

5. Scheduler

Tres triggers — un único punto de entrada

Todo converge en ejecutar_conciliacion(). Independientemente del trigger, la deduplicación por hash en Redis garantiza que el mismo CAMT.053 nunca se procese dos veces aunque llegue por dos canales simultáneamente.

Trigger	Cuándo	Configuración	Uso típico
Cron diario	Hora fija configurable	CRON_HORA=18, CRON_MINUTO=30, CRON_TZ=Europe/Madrid	Banco con entrega por webhook
Webhook banco	Al generar el CAMT.053	POST /webhook/camt053 con X-Banco-Token	Banco con notificaciones push
Watcher SFTP	Cada 15 minutos	SFTP_HOST, SFTP_USER, SFTP_INBOX	Banco con entrega por SFTP
Trigger manual	On-demand	POST /admin/conciliar-ahora (solo red interna)	Ejecutar tras un error

Deduplicación: cada CAMT.053 se hashea con SHA-256 y se guarda en Redis con TTL de 24 horas. Si el mismo archivo llega por webhook y por SFTP en la misma hora, solo se procesa una vez. El segundo intento se omite en microsegundos.

6. Base de datos PostgreSQL

5 tablas · triggers automáticos · vistas analíticas

Tabla	Filas	Descripción
mensajes_iso20022	1 por mensaje	XML crudo inmutable — fuente de verdad. Append-only.
conciliacion_runs	1 por ejecución	Resumen ejecutivo: tasa, importes, narrativa LLM.
resultados_conciliacion	1 por transacción	Cada decisión documentada: estado, método, motivo, diffs.
audit_events	N por ejecución	Log granular de cada llamada a herramienta del agente.
excepciones_abiertas	1 por excepción	Alimentada por trigger automático. SLA y asignación.

Trigger automático de excepciones

El trigger `crear_excepcion_automatica()` se ejecuta en PostgreSQL tras cada INSERT en `resultados_conciliacion`. Si el estado es `RECHAZADO`, `SIN_MATCH`, `EXCEPCION_PARCIAL` o `MATCH_FUZZY` con revisión requerida, crea automáticamente una fila en `excepciones_abiertas` con prioridad y fecha de vencimiento SLA calculadas según el tipo:

Estado	Prioridad	SLA (días hábiles)	Acción por defecto
RECHAZADO	1 — Alta	1 día	Gestionar rechazo con el banco
SIN_MATCH	1 — Alta	2 días	Verificar si está en tránsito o rechazado
EXCEPCION_PARCIAL	2 — Media	3 días	Investigar diferencia con el banco
EN_TRANSITO	3 — Baja	2 días	Verificar en extracto del día siguiente
MATCH_FUZZY	2 — Media	1 día	Confirmar o descartar el match propuesto

Vistas analíticas incluidas

Vista	Descripción
v_kpis_diarios	KPIs agregados por día: tasa media, totales, importes, duración
v_excepciones_aging	Excepciones abiertas con días transcurridos y estado SLA
v_efectividad_motores	Tasa de éxito por método de matching (UETR, E2E, fuzzy, embedding)
v_resumen_semanal	Resumen semanal para el CFO: pagos, tasa, importes, tiempo medio

7. API del dashboard

9 endpoints REST — documentación automática en /docs

Método	Ruta	Descripción
GET	/api/kpis?periodo=hoy semana mes	KPIs + donut + historial en una llamada
GET	/api/excepciones	Lista paginada con filtros (estado, tipo, prioridad)
PATCH	/api/excepciones/{id}/resolver	Marcar excepción como resuelta con nota obligatoria
GET	/api/runs	Historial de ejecuciones del agente
GET	/api/runs/{run_id}	Detalle completo: resumen + todos los resultados
GET	/api/audit/{run_id}	Audit trail granular para revisión regulatoria
GET	/api/pagos/{end_to_end_id}	Trazabilidad completa de un pago por EndToEndId
GET	/api/kpis/semana	Histórico diario de 7 días para el mini chart
GET	/api/health/detalle	Estado de PostgreSQL, Redis y último run

FastAPI genera automáticamente la documentación Swagger en <http://localhost:8000/docs> y la especificación OpenAPI en `/openapi.json`. Puedes importarla directamente en Postman o Insomnia para pruebas.

Integrar en agente_completo.py

```
# Añadir al final de agente_completo.py:
from api_dashboard import router_dashboard
app.include_router(router_dashboard, prefix='/api')

# Ejemplo de llamada desde el dashboard React:
# GET /api/kpis?periodo=hoy
# → { tasa: 94.7, kpis: [...], donut: [...], historial: [...] }
```

8. Frontend React

Dashboard.jsx — panel operativo para el equipo de tesorería

El dashboard está construido en React con Tailwind CSS. Se conecta a la API FastAPI y cae a datos de demo cuando la API no está disponible. El diseño es dark-mode nativo optimizado para uso en pantalla de escritorio.

Componente	Descripción
KPICard	Tarjeta de KPI con valor, color semántico y delta vs período anterior
DonutChart	Gráfico donut SVG nativo con leyenda de distribución de estados
ChipEstado	Badge de color por tipo de excepción (Excepción / Rechazo / Sin match / ...)
PrioDot	Indicador de prioridad Alta/Media/Baja con punto de color
SLABadge	Estado del SLA: VENCIDA (rojo) / Vence hoy (ámbar) / En plazo (gris)
Modal	Modal de resolución con campo de nota obligatoria y confirmación
Toast	Notificación temporal de éxito/error en esquina inferior derecha
useKPIs()	Hook que llama a /api/kpis con fallback automático a datos demo

Instalación rápida

```
npm create vite@latest dashboard -- --template react
cd dashboard && npm install
# Instalar Tailwind: https://tailwindcss.com/docs/guides/vite
cp Dashboard.jsx src/App.jsx

# Variables de entorno
echo 'VITE_API_URL=http://localhost:8000' >> .env
echo 'VITE_SCHEDULER_URL=http://localhost:8001' >> .env

npm run dev # http://localhost:5173
```

9. Infraestructura Docker

Levantar todo el sistema con un solo comando

docker-compose.yml orquesta 5 servicios con healthchecks entre ellos. El agente no arranca hasta que Postgres responde a pg_isready. El scheduler no arranca hasta que el agente responde a /health.

Servicio	Imagen	Puerto	Healthcheck	Depende de
postgres	postgres:16-alpine	5432	pg_isready	—
redis	redis:7-alpine	6379	redis-cli ping	—
agente	build: .	8000	GET /health	postgres, redis
scheduler	build: .	8001	GET /health	agente, postgres, redis
nginx	nginx:alpine	80/443	—	agente, scheduler

Dockerfile multi-stage

Stage	Base	Contenido	Para qué
deps	python:3.11-slim	libxml2, libpq, pip install	Cache de dependencias
development	deps	Todo el código + hot-reload	Desarrollo local
production	deps	Solo archivos .py necesarios + user	Producción (~200MB)

Seguridad: el stage de producción corre con el usuario 'treasury' sin privilegios. nginx.conf bloquea las rutas /admin/ a redes privadas (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) y aplica rate limiting estricto al webhook del banco (10 req/min).

10. Operación — comandos del día a día

Makefile incluido con todos los shortcuts

Arranque inicial (primera vez)

```
# 1. Clonar el repositorio y entrar
git clone && cd conciliacion-iso20022

# 2. Configurar variables de entorno
cp .env.example .env
# Editar .env: añadir ANTHROPIC_API_KEY y WEBHOOK_SECRET

# 3. Levantar todo
make up
# → Postgres + Redis + Agente :8000 + Scheduler :8001 + Nginx :80

# 4. Verificar estado
make ps
curl http://localhost:8000/health
curl http://localhost:8001/health
```

Operación diaria

Comando	Descripción
make logs	Ver logs de todos los servicios en tiempo real
make logs-agente	Ver solo los logs del agente LangGraph
make conciliar-ahora	Disparar conciliación manual con fixtures de prueba
make shell-postgres	Abrir consola psql para consultas directas
make backup-db	Backup comprimido en backups/treasury_db_YYYYMMDD.sql.gz
make rebuild	Reconstruir imágenes Docker sin cache y redeployar

Queries SQL operativas frecuentes

```
-- Excepciones abiertas hoy por prioridad
SELECT * FROM v_excepciones_aging WHERE fecha = CURRENT_DATE;

-- Trazabilidad completa de un pago
SELECT r.*, cr.created_at AS run_iniciado
FROM resultados_conciliacion r
JOIN conciliacion_runs cr ON cr.run_id = r.run_id
WHERE r.end_to_end_id = 'E2E-TX-001';

-- KPI de tasa de los últimos 30 días
SELECT fecha_conciliacion, tasa_media_pct
FROM v_kpis_diarios
WHERE fecha_conciliacion >= CURRENT_DATE - 30;

-- Rechazos por código en la última semana
SELECT codigo_rechazo, COUNT(*), SUM(importe_pacs_eur)
FROM resultados_conciliacion
WHERE estado = 'RECHAZADO'
AND fecha_conciliacion >= CURRENT_DATE - 7
```

```
GROUP BY codigo_rechazo ORDER BY 2 DESC;
```

11. Roadmap y próximos pasos

Tres fases para escalar el sistema

Fase 1 — Producción (meses 0–3)

- Conectar con credenciales reales del banco (JPMorgan API, HSBC, Plaid)
- Activar SOC 2 Type I (proceso de 6 meses — iniciar ya)
- Tests de integración con 50+ casos edge del motor de matching
- Dashboard: añadir filtro por entidad bancaria y exportación a Excel
- Monitorización: alertas Datadog cuando tasa < 90% o excepción P1 sin resolver

Fase 2 — Escala (meses 3–9)

- CAMT.052 intradiario: conciliación en tiempo real durante el día
- Multi-entidad: soporte para 10+ entidades legales en paralelo
- Integración ERP: cerrar el loop contable automáticamente en SAP/NetSuite
- Motor FX: conversión de importes multi-moneda con tasas en tiempo real
- API pública para que los clientes conecten su propio TMS

Fase 3 — Plataforma (meses 9–18)

- Agente autónomo: propone asientos contables de ajuste para excepciones recurrentes
- Matching ML: modelo entrenado con datos históricos del cliente (supera embeddings)
- Multi-banco coordinado: agentes PACS comunicándose entre corporates y bancos
- White-label: dashboard configurable por cliente con su branding
- Certificación ISO 27001 para enterprise accounts

El sistema está listo para producción. Con 'make up' tienes toda la infraestructura corriendo. El paso inmediato es reemplazar los fixtures de prueba por las credenciales reales del banco en `bank_integration_layer.py` y ejecutar 'make conciliar-ahora' con el primer CAMT.053 real.